Secure Coding Review Task Report

- **Target Project:** PyGoat Web Application
- **Testing Tool Used:** Bandit Static Analyzer
- **Submitted by:** Sameh Hassan
- **University:** Egyptian Chinese University (ECU)

Executive Summary: I used the Bandit static analyzer to scan the source code of the PyGoat application (Python/Django). The tool scanned around 3124 lines of code and successfully identified multiple vulnerabilities. In this report, I have focused on the top 4 critical and medium vulnerabilities that could seriously compromise the application's security.

Detailed Findings & Remediation:

1. Weak Password Hashing (MD5)

- **Severity Level:** High - CWE-327
- **File Location:** introduction/mitre.py (Line 161)
- **Vulnerability Description:** The developer used the MD5 algorithm to hash passwords (password = md5(password.encode()).hexdigest()). MD5 is severely outdated and extremely vulnerable to collision attacks, making it easy for attackers to crack and retrieve the original passwords.
- **Remediation:** MD5 should be completely removed. We must use a modern, cryptographically strong algorithm like SHA-256, or preferably rely on dedicated password hashing libraries such as bcrypt.

2. OS Command Injection

- **Severity Level:** High - CWE-78
- **File Location:** introduction/mitre.py (Line 233)
- **Vulnerability Description:** The code utilizes the subprocess.Popen function with the argument shell=True. This is a critical security flaw because it allows a malicious user to inject arbitrary OS commands (Linux/Windows) that will execute directly on the host server.
- **Remediation:** Always set shell=False. Furthermore, when passing commands to the system, they should be passed as a List of strings rather than a single concatenated string to prevent argument injection.

3. Debug Mode Enabled in Production

- **Severity Level:** High - CWE-94
- **File Location:** dockerized_labs/broken_auth_lab/app.py (Line 123)
- **Vulnerability Description:** The application is configured to run with debug=True. If an error occurs, the server will expose sensitive source code, file paths, and environment variables to the public. An attacker could also exploit the Werkzeug debugger to achieve Remote Code Execution (RCE).
- **Remediation:** Before deploying the application to a production environment, this variable must be explicitly set to debug=False.

4. SQL Injection

- **Severity Level:** Medium - CWE-89
- **File Location:** introduction/views.py (Line 158)
- **Vulnerability Description:** The code constructs a database query by directly concatenating user input (username and password) into the SQL string. This makes the application highly vulnerable to SQL Injection, allowing an attacker to bypass authentication entirely.
- **Remediation:** Stop using direct string concatenation for SQL queries. The code should be refactored to use Parameterized Queries or rely entirely on the framework's built-in ORM, which automatically sanitizes user inputs.